

Exercise 2 (Summer 2026)

6.0 VU Advanced Database Systems

Information

General

Work through the exercises below and write a report on your solutions. Submit the report and your source files as a **single ZIP (.zip)** file on TUWEL. The ZIP (.zip) file must contain:

- one PDF (.pdf) file,
- the JupyterLab notebook (.ipynb) files for the MapReduce and Spark exercises.

The ZIP (.zip) file must be at most 20 MB in size. After submitting the files, you need to register for an exercise interview. **You can only receive points for this exercise if you attend the interview.** You are expected to discuss your solutions and answer questions regarding them in your report. **We will not accept any handwritten reports!**

For MapReduce and Spark exercises, we expect that you attempt to run your solutions through JupyterHub web interface in the cluster. Instructions on how to access the web interface are detailed below.

This exercise sheet contains 5 exercises, with a total of 15 points achievable.

Deadlines

at latest May 12th 9:00 Upload your submission on TUWEL
Do not forget! ⇒ Register for an exercise interview in TUWEL

Exercise Interviews

During the exercise interview, the correctness of your solution as well as your understanding of the underlying concepts will be assessed. The scoring of your submission is primarily based on your performance at the exercise interview. Therefore it is possible (in extreme cases) to get 0 points even though the submitted solution was technically correct.

Please be punctual for your exercise interview. Otherwise we cannot guarantee that your full solution can be graded in your assigned time slot. **Remember to have your student id with you for the exercise interview.** It is not possible to receive a score for your solution without showing your id, as we cannot confirm your identity.

TUWEL Forum

You can use the course forum on TUWEL for clarifying questions regarding the exercise sheets. Please do not post your solutions (even partial) on the forum.

For technical questions, or in the case of technical problems, please use the *Technical Cluster Support forum* on TUWEL, as the cluster team will monitor it.

Connecting to JupyterHub at LBD Cluster

To work on Spark exercise, you are expected to log in to JupyterHub at the LBD Cluster via TU Wien Single Sign-On: <https://lbd.tuwien.ac.at>

Please note: The connection to the dataLab portal can only be established from inside the TU Wien network. If you want to connect from outside the campus, you have to connect to the TU Wien network via VPN first. (<https://www.it.tuwien.ac.at/en/services/network-and-servers/tuvpn>)

Exercises

Exercise 1 (Distributed Joins) [3 points] Find the distributed execution strategy for the joins that minimizes communication cost (number of bytes transferred in this case). To do this, compute the communication costs for the strategies presented in the lecture (slide 46 onward of the DDBMS slides) and compare the results. For the whole exercise assume a distributed database with 4 sites and three relations: **Authors**, **Books**, and **Libraries**. The details of the scenario are described in Figure 1. You can assume that all relations are non-fragmented and all records (and attributes) are fixed-size. **Note that only the important fields are listed below.** Assume there are other fields to get to the total record sizes listed. Record and attribute sizes are given in bytes. Additionally, note that there are no foreign key constraints between these tables.

Relation	Site	# Records	Record Size (byte)
Authors	1	4 000	450
Books	2	50 000	400
Libraries	3	10 000	750

Relation	Attribute	Size
Authors	authid	12
Authors	name	58
Books	author	10
Books	title	30
Books	library	10
Libraries	libid	10

Figure 1: Scenario for Exercise 1

- (a) Compute the communication cost of all strategies from the slides (and listed below) and find the strategy that minimizes communication cost for the Query 1 given below if we want the result at site 4. State all assumptions you used for cardinality estimation during the calculations. Assume a selectivity of $\frac{1}{9500}$ for the join.

Strategies:

- Send both tables to site 4 and join there.
- Send **Authors** to site 2, join there and send the result to site 4.
- Symmetrically: Send **Books** to site 1, join there and send the result to site 4.
- Send only the join attributes of **Authors** to site 2, semi join with **Books**, and send the result back to site 1 to compute the full join. Finally transfer the full join to site 4.
- The semi-join strategy in the opposite direction.

$$\pi_{name,title}(Authors \bowtie_{authid=author} Books) \quad (1)$$

Hint: For the result size of a semi-join $A \bowtie B$, you can use $\min\{|A \bowtie B|, |A|\}$ as a worst-case estimate

- (a*) Adapt the strategies above in such a way, that you always project to only the necessary columns as early as possible. How does the amount of data that needs to be transferred change?
- (b) Now consider how to generalize the strategies of task (a*) above to situations with more than 2 relations and find the strategy with optimal communication cost for the Query 2 given below. Again we want the result at site 4. Again, assume a selectivity of $\frac{1}{9500}$ for the join between **Authors** and **Books**. For the outside join assume a selectivity of $\frac{1}{4000}$.

Having computed the costs of the inner join, you can skip calculating the complete communication costs for all those strategies that cannot result in minimal communication costs. If you do so, *explain why you can skip the respective calculations*.

$$Libraries \bowtie_{libid=library} \pi_{name,library}(Authors \bowtie_{authid=author} Books) \quad (2)$$

Exercise 2 (Cost of MapReduce) [2 points] Given in this exercise is a MapReduce program for constructing a **Top-K user similarity graph** from a user-item interaction graph. Let $G = (U \cup V, E)$ be a user-item interaction graph where U is the set of users, V is the set of items (e.g., movies, books, products), and $(u, v) \in E$ where $u \in U, v \in V$. Assume that the set of users U are ordered by their indices. The graph contains no edges within U or within V , i.e., G is a bipartite graph.

The goal is to construct a weighted user-user similarity graph, where:

- Two users u_1 and u_2 are connected by an edge if they share common neighbors in V .
- Only the top-k most similar users are retained based on their similarity scores (computed based on the number of common items).

Mapper 1: *Input:* a split of edges $E' \subseteq E$

For each edge $(u, v) \in E'$:

Emit (v, u) (reverse index: item $\rightarrow$ user)

Reducer 1: *Input:* (key = v , values = list of users connected to v)

Identity reducer

Mapper 2: *Input:* (key = v , list of users connected to v)

Let $\text{Users}(v)$ be all users connected to item v .

For every pair of users (u_1, u_2) where $u_1 < u_2$ in $\text{Users}(v)$:

Emit $((u_1, u_2), 1)$

Reducer 2: *Input:* (key = (u_1, u_2) , values = list of 1s)

Emit $((u_1, u_2), \sum \text{values})$

Mapper 3: *Input:* (key = (u_1, u_2) , score)

Emit $(u_1, (u_2, \text{score}))$

Emit $(u_2, (u_1, \text{score}))$

Reducer 3: *Input:* (key = u , values = list of candidates and their similarity scores)

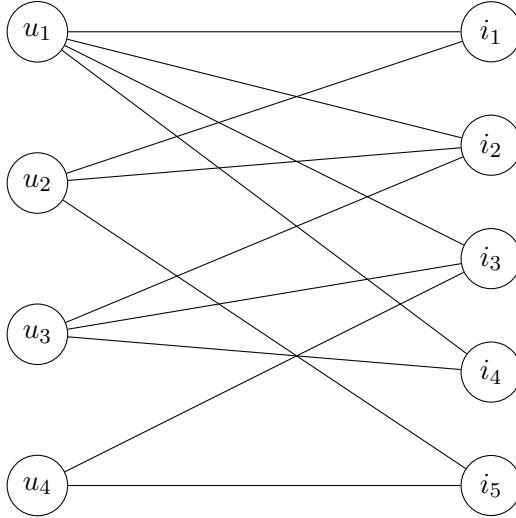
Let $\text{TopK}(u)$ be the set of the k most similar users v to u , each with similarity score s .

Resolve ties arbitrarily if there are more than k users with the same similarity score.

For each user v in $\text{TopK}(u)$:

Emit $((u, v), s)$

(a) Consider the following user-item interaction graph:



Compute the given mapreduce algorithm with the edges of this graph as an input and $k = 2$. Show the intermediate results of each map phase and reduce phase of the three stages:

- Mapper 1 output
- Reducer 1 output
- Mapper 2 output
- Reducer 2 output
- Mapper 3 output
- Reducer 3 output

If mapper and reducer outputs are identical, you may omit repetition and refer to the previous phase.

(b) Analyze these cost measures for each stage of the MapReduce program:

- Communication cost
- Replication rate
- Reducer size

Describe these values depending on the input graph $G = (U \cup V, E)$. The degree of a node (the number of edges connected to this node) v , is denoted by $\deg(v)$. For nodes $u_1, u_2 \in U$, let $\text{CI}(u_1, u_2)$ be the set of common items between users u_1 and u_2 . For a node $u \in U$, let $S(u)$ be the set of users with common items with u .

Recall that the replication rate in a round is determined based on the input and output sizes of the mapper in that round. You may express the reducer size in terms of cardinalities rather than data size (i.e., number of bytes).

Exercise 3 (MapReduce Algorithms) [2 points] You are asked to implement the relational algebra expression and matrix computation defined below as informal MapReduce algorithms. It is enough for each of them to give a complete description of what the mapper and reducer functions are doing in plain text. Mathematical notation is preferred where

applicable. There is no need for pseudo-code. Make it clear what the input or output of each function precisely is. You may use “Emit” when a function adds something to its output.

- (a) Given three relations $X(a, b)$ and $Y(a, b, c)$ and $Z(b, c, d)$, operation F_1 that gives the new output relation, provide the MapReduce algorithm to compute $F_1(X, Y, Z)$ defined as follows:

$$F_1(X, Y, Z) = \gamma_b SUM(a)(\sigma_{a>b}(X) - \pi_{a,b}(Y \ltimes Z))$$

where γ_b is the group-by operator that groups the tuples by the value of attribute b , σ is the selection operator, and π is the projection operator.

Assume that set semantic holds for this exercise, i.e. no duplicates are possible.

- (b) Given a matrix M , its min-max normalized matrix M' is defined as follows:

$$M'_{ij} = \frac{M_{ij} - \min(M_i)}{\max(M_i) - \min(M_i)}$$

where $\min(M_i)$ and $\max(M_i)$ are the minimum and maximum values in the i -th row of the matrix M , respectively. I and J are the number of rows and columns of the matrix M , respectively. In the case where $\max(M_i) = \min(M_i)$, we define $M'_{ij} = 0$ for all j . Provide a MapReduce algorithm to compute M' from M .

Exercise 4 (MapReduce in Practice) [3 points] For this exercise, you are tasked with writing your own MapReduce program in Python and to run it on the cluster on the provided datasets. The dataset for this exercise is based on the stackexchange data dump available here (<https://archive.org/details/stackexchange>). It contains the data of users, posts and comments. You will work on the JupyterLab notebook provided on TUWEL to write and execute MapReduce jobs on the JupyterLab web editor. You can find the dataset on the cluster under the following locations:

Local file system

/home/adbs_shared/Ex_2/stackexchange/users.csv
 /home/adbs_shared/Ex_2/stackexchange/posts.csv
 /home/adbs_shared/Ex_2/stackexchange/comments.csv

- (a) In Stack Exchange, users can ask questions, answer questions, and comment on both questions and answers. Write a MapReduce job with `posts.csv` and `comments.csv` as inputs and following output:

(user_questioned, user_answered, user_commented)

where each triple represents a complete question-answer-comment relationship: a user who asked a question, a user who answered that question, and a user who commented on the corresponding answer. Comments directly on questions can be ignored.

In `posts.csv`, the user ids are stored in `owneruserid`. Column `posttypeid` stores whether a post is a question (`posttypeid=1`) or an answer (`posttypeid=2`). For answer posts, the column `parentid` stores the id of the corresponding question. In `comments.csv`, the answer corresponding to the comment is stored in `postid` and the user who commented is stored in `userid` (You may ignore the comments where the `userid` is empty).

- (b) Write a MapReduce job with `posts.csv` and `users.csv` as input and following output: for each user compute the total number of question and answer posts they have created and output the 100 users with highest reputation.

Make sure the output has the following schema:

`displayname, reputation, question_count, answer_count`

In `posts.csv`, the user ids are stored in `owneruserid`. Column `posttypeid` stores whether a post is a question (`posttypeid=1`) or an answer (`posttypeid=2`). In `users.csv`, the user ids are stored in `id` and their reputations are stored in `reputation`. In case that there are multiple users with the same reputation, you may resolve these conflicts randomly, i.e. pick one of them arbitrarily.

Use a combiner function where possible to reduce the communication cost. Make sure that your program correctly deals with the header, data types, and possible missing values.

Exercise 5 (Spark) [5 points] For this exercise, you will work on the JupyterLab notebook, which is provided on TUWEL, and solve the tasks listed therein. These tasks, in addition to writing Spark code, require you to analyse various query plans and to reason about them. To get a deeper understanding, and look up the types and definitions of various functions, we recommend that you visit the Spark¹ and SparkSQL² documentation. You can find the data on the cluster under the following location:

Local file system

```
/home/adbs_shared/stackexchange/badges.csv
/home/adbs_shared/stackexchange/comments.csv
/home/adbs_shared/stackexchange/posts.csv
/home/adbs_shared/stackexchange/postlinks.csv
/home/adbs_shared/stackexchange/users.csv
/home/adbs_shared/stackexchange/votes.csv
```

- a) You are given a series of queries in SQL literal syntax (resp. Dataframe), and are asked to provide an equivalent Dataframe (resp. SQL literal syntax) version of it.
- b) You are given two independent Spark applications. Each application consists of a sequence of Spark jobs, and each job consists of multiple transformations followed by actions. Two optimization blocks have been introduced in an attempt to improve the performance of these applications. Based on the provided programs, answer the following questions:
- (i) Ignoring the two optimization blocks, identify the stages and their boundaries for each Spark job: `user_scores`, `tag_scores`, and `user_comment_scores`. Clearly explain your reasoning using Spark's processing model.
 - (ii) For Application 1, analyze and discuss the impact of the two optimization blocks on the performance of the application. Explain whether they improve

¹<https://spark.apache.org/docs/latest/rdd-programming-guide.html>

²<https://spark.apache.org/docs/latest/sql-programming-guide.html>

performance, and justify your answer.

- (iii) For Application 2, analyze and discuss the impact of the two optimization blocks on the performance of the application. Again, explain whether they improve performance, providing justification.
- c) Revisit the algorithm for constructing a **Top-K user similarity graph** from a user-item interaction graph, which was introduced in Exercise 2. Translate the algorithm into Spark RDD transformations and actions and implement it using the pyspark library.
- d) You are given a dataset of retail transactions, where each transaction consists of a list of items bought together. Using Spark RDD transformations and actions, implement an algorithm to identify the top-K most frequently co-purchased item pairs.